Domain-Driven Design Basics

one language, small aggregates

Watch the 4-min video

In one line: DDD models complex business rules with a shared **ubiquitous language**, entities and value objects, and **aggregates**: clusters with one root that enforces the rules, changed in one transaction, and linked to other aggregates by domain events with eventual consistency.

1 Lost in translation

The business says *booking*, the code says *ReservationRecord*, the database says *slot_entry*. Every translation is a chance for a bug and slows every conversation. Domain-driven design (Eric Evans, 2003) starts by fixing the language, then gives you a few building blocks for modelling rich business rules.

2 Ubiquitous language

Developers and domain experts agree on the words for business concepts and use exactly those words in conversation, documents and code. If the business says *order line*, the class is `OrderLine`. When the words change, the code changes too. The language is only valid inside one context, which leads to bounded contexts (lesson 22).

3 Building blocks

Block	What it is	Example
Entity	has an identity that stays the same while its data changes	Customer #42, Order #1001
Value object	no identity; defined by its values; immutable, replaced not edited	Money(25.00, EUR), Address
Aggregate	cluster changed as one unit, with one root as the only entry point	Order + its OrderLines
Domain event	a fact that happened, named in past tense	OrderPlaced, PaymentFailed
Repository	loads and saves whole aggregates, hiding the database	OrderRepository.get(id)

Two addresses with the same street and city are simply equal; two customers with the same name are not the same customer. That is the entity / value object difference in one sentence.

4 Aggregates and transactions

An **aggregate** is the unit of consistency. Its root enforces the **invariants**, the rules that must always be true, such as "total ≤ credit limit". Because outside code can only call the root, no one can bypass the rules by editing a child directly. Vaughn Vernon's rules of thumb:

- **One transaction changes one aggregate.** Keep aggregates small so transactions are short and conflicts rare.
- **Reference other aggregates by ID**, not by holding the object.

- **Use domain events** to update other aggregates afterwards, with eventual consistency.
- **One repository per aggregate**, not per table.

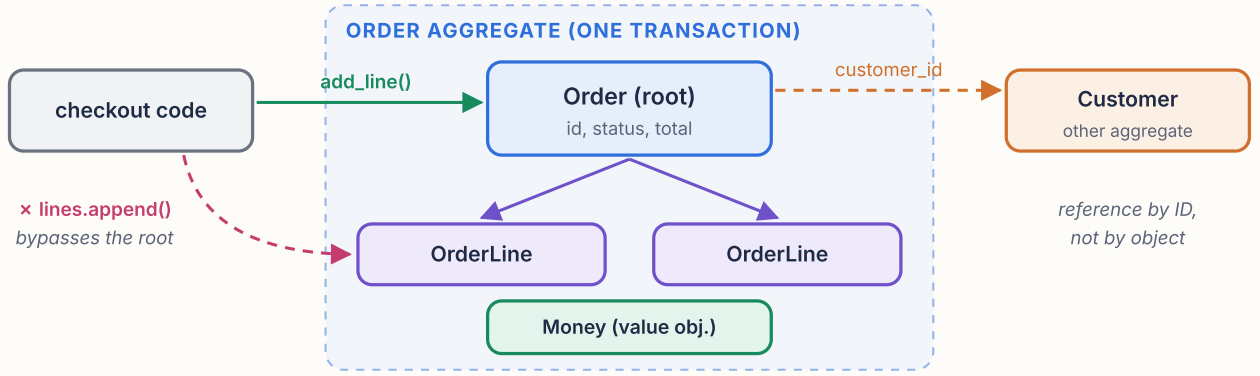


Figure 1. Outside code holds a reference only to the root. The root checks the invariants for everything inside.

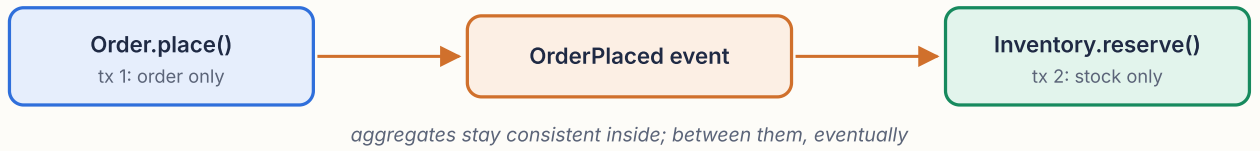


Figure 2. A change that spans aggregates becomes two transactions joined by a domain event.

Why it helps system design: aggregates show where transactions end; domain events show where services can be decoupled; language changes show where contexts and services split. DDD pays off when the business rules are rich; for simple CRUD it is overkill.

WORKED EXAMPLE: THE ORDER AGGREGATE

`Order` is the root; `OrderLine`s live inside. Invariants: the total can't exceed the customer's credit limit, and a shipped order can't change. Code can't call `order.lines.append(line)`; it must call `order.add_line(product, qty)`. The root checks the status is not SHIPPED, computes the new total, rejects it if it exceeds the limit, and otherwise adds the line. `OrderRepository.save(order)` writes the order and its lines in **one transaction**. Reserving stock happens later, when inventory handles the `OrderPlaced` event.

COMMON MISTAKES

- Huge aggregates (Customer with all orders) causing lock contention and slow loads.
- Updating two aggregates in one transaction "for convenience".
- Anaemic models: getters and setters only, with the rules scattered in services.
- Using DDD tactics everywhere, even for simple CRUD screens.

5 Key terms

Ubiquitous language: shared words used in talk and code alike

Entity: object with a stable identity

Value object: immutable, compared by value

Aggregate root: the single entry point that enforces invariants

Invariant: business rule that must always hold

Domain event: past-tense fact, e.g. OrderPlaced

6 Interview questions

Q1. What is the difference between an entity and a value object? BASIC
An entity has an identity that persists as its data changes; a value object has no identity, is immutable and is equal to any other with the same values.

Q2. What is the ubiquitous language? BASIC
Words for domain concepts agreed by developers and domain experts and used unchanged in conversation, documents and code, within one bounded context.

Q3. What is an aggregate and why does it matter for transactions? SCENARIO
A cluster with one root that enforces its invariants; it is the unit of consistency, so one transaction should change one aggregate, and aggregates should be small.

Q4. Placing an order must also reserve stock. How do you model it? SCENARIO
Order and Inventory are separate aggregates: commit the order, publish OrderPlaced, and let inventory reserve stock in its own transaction, with a compensation if it fails.

Q5. Why reference other aggregates by ID instead of object? TRICKY
It keeps aggregate boundaries and transactions clear, avoids loading large object graphs, and lets the aggregates live in different services later.

Q6. What links aggregates if not transactions? TRICKY
Domain events such as OrderPlaced, handled asynchronously, giving eventual consistency between aggregates.

Next: SD-L-004 · Bounded Contexts backendroadmap.com